

**SIMATS ENGINEERING
ASSESSMENT TOOL 2**

COURSE NAME: Design and Analysis of Algorithms
COURSE CODE: CSA0619
NAME: GAYATHERI S
REG NO: 192521306

COURSE OUTCOME COVERED

CO2: Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)

Assessment Tool Weightage: 20%

Annexure A

Scenario Based Assessment

Q12. A delivery company maintains GPS coordinates of delivery points and computes distances between every pair to find the closest drop location. As the number of locations increases, delays are observed.

- a) Explain how the brute-force closest pair algorithm operates in this scenario.
- b) Derive the time complexity of computing all pair distances.
- c) Analyse why the performance degrades with large datasets.

SOLUTION

A delivery company stores the GPS coordinates of n delivery points. To identify the closest pair of delivery locations, the brute-force approach calculates the distance between every possible pair of points and keeps track of the smallest distance found. This method is simple and guarantees the correct answer, but its performance decreases rapidly as the number of locations increases.

a) Operation of the Brute-Force Closest Pair Algorithm

ALGORITHM DEFINITION

An **algorithm** is a finite set of well-defined instructions or steps designed to solve a problem or perform a specific task.

Characteristics of an Algorithm

- **Input:** Takes zero or more inputs.
- **Output:** Produces at least one output.
- **Definiteness:** Each step is clear and unambiguous.
- **Finiteness:** Terminates after a finite number of steps.
- **Effectiveness:** Each step is basic and feasible to perform.

In Simple Words: An algorithm is a step-by-step procedure or formula that solves a problem and gives the correct result.



Problem Identification

The objective is to find the two delivery points that have the minimum distance between them.

Suppose the GPS coordinates are represented as:

$$P_i = (x_i, y_i)$$

For every pair of delivery points P_i and P_j , the Euclidean distance is calculated using:

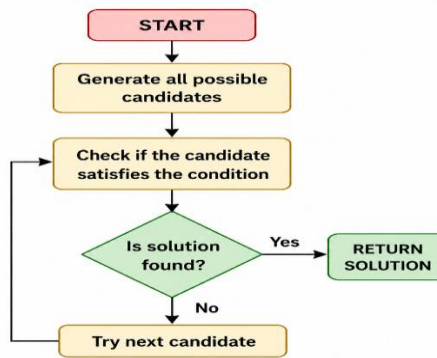
$$d(P_i, P_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

Working Procedure

The brute-force algorithm operates as follows:

1. Store all delivery locations as GPS coordinate pairs (x, y) .
2. Initialise the minimum distance as infinity.
3. Select the first delivery point.
4. Compare it with every point that comes after it.
5. Calculate the distance between each pair.
6. If the calculated distance is smaller than the current minimum, update the minimum distance.
7. Store the corresponding pair of delivery locations.
8. Repeat the process until every unique pair has been checked.
9. Return the pair having the smallest distance.

GENERAL FLOW OF BRUTE FORCE



Example

Consider four delivery points:

$A(2, 3), B(5, 7), C(3, 4), D(10, 8)$

The algorithm checks:

- $A - B$
- $A - C$
- $A - D$
- $B - C$
- $B - D$
- $C - D$

The distance of every pair is calculated and compared. The pair with the minimum distance is selected as the closest drop location pair.

Pseudocode

```
main.py  [Icons] [Share] [Run]
1 print("Start small. Ship something.")BruteForceClosestPair(points):
2     min_distance = infinity
3     closest_pair = None
4
5     for i = 0 to n-2:
6         for j = i+1 to n-1:
7             distance = calculate_distance(points[i], points[j])
8
9             if distance < min_distance:
10                 min_distance = distance
11                 closest_pair = (points[i], points[j])
12
13     return closest_pair, min_distance
```

The use of $j = i + 1$ is important because it avoids comparing a point with itself and prevents checking the same pair twice.

b) Derivation of Time Complexity

For n delivery locations, the algorithm uses two nested loops.

The first location is compared with:

$$n - 1$$

locations.

The second location is compared with:

$$n - 2$$

locations, and so on.

Therefore, the total number of distance calculations is:

$$(n - 1) + (n - 2) + (n - 3) + \dots + 2 + 1$$

This is the sum of the first $n - 1$ natural numbers:

$$\frac{n(n - 1)}{2}$$

Expanding:

$$\frac{n^2 - n}{2}$$

Ignoring constants and lower-order terms for asymptotic analysis:

$$T(n) = O(n^2)$$

Therefore, the time complexity is:

$$\boxed{O(n^2)}$$

Number of Comparisons for Different Dataset Sizes

Number of Locations (n)	Number of Pair Comparisons
10	45
100	4,950

Number of Locations (n)	Number of Pair Comparisons
1,000	499,500
10,000	49,995,000
100,000	4,999,950,000

This demonstrates that the number of comparisons increases quadratically.

Space Complexity

If only the current minimum distance and closest pair are stored:

$$O(1)$$

additional space is required, excluding the input array containing the GPS coordinates.

c) Analysis of Performance Degradation with Large Datasets

The main reason for performance degradation is the quadratic time complexity $O(n^2)$.

As the number of delivery locations increases, the number of possible pairs increases much faster. For example, increasing the number of locations from 1,000 to 10,000 increases the number of pair comparisons from approximately 0.5 million to 50 million.

The following factors contribute to the delay:

1. Rapid Increase in Pair Comparisons

Every new location must be compared with almost all existing locations. Therefore, adding more delivery points creates a large number of additional distance calculations.

2. Repeated Distance Calculations

The algorithm repeatedly performs operations such as subtraction, multiplication, addition and square-root calculation for each pair. These computations become expensive when millions or billions of pairs are involved.

3. Poor Scalability

The $O(n^2)$ growth makes the brute-force method unsuitable for very large delivery networks. A small increase in input size can cause a significant increase in execution time.

4. Increased Computational Resources

Large datasets require more CPU processing and may also increase memory and data-access overhead, especially if all pairwise distances are stored instead of calculated individually.

5. Real-Time Delivery Requirements

In a real delivery system, locations may be updated continuously using GPS. Recalculating all pairs whenever new locations are added can introduce delays, making the brute-force approach unsuitable for real-time applications.

6. Lack of Algorithmic Efficiency

The brute-force algorithm does not eliminate unnecessary comparisons. It examines every possible pair even when many pairs are obviously far apart. More efficient methods can reduce the number of comparisons significantly.

Optimisation Analysis

The brute-force method is easy to understand and implement and guarantees the correct closest pair. However, it is not efficient for large datasets. A more efficient divide-and-conquer closest-pair algorithm can reduce the time complexity from:

$$O(n^2)$$

to:

$$O(n \log n)$$

This provides a significant improvement for large-scale delivery systems.

Comparison

Method	Time Complexity	Suitability
Brute Force	$O(n^2)$	Small datasets
Divide and Conquer	$O(n \log n)$	Large datasets

The brute-force closest pair algorithm calculates the distance between every unique pair of GPS delivery locations and selects the pair with the minimum distance.

For n locations, it performs:

$$\frac{n(n-1)}{2}$$

distance calculations, resulting in a time complexity of:

$$\boxed{O(n^2)}$$

Although the method is simple, accurate and easy to implement, its performance degrades significantly as the dataset grows because the number of pairwise comparisons increases quadratically. Therefore, for large-scale or real-time delivery applications, an optimised $O(n \log n)$ closest-pair algorithm is more appropriate.

Rubric Alignment

- Problem Identification – Exemplary: Clearly identifies the closest-pair problem, GPS distance calculation and source of delay.
- Debugging/Analysis – Exemplary: Systematically analyses the pairwise comparison process and derives the complexity mathematically.
- Optimisation – Exemplary: Identifies the limitation of $O(n^2)$ and proposes a more efficient $O(n \log n)$ approach.
- Analysis – Exemplary: Provides logical reasoning, numerical examples and scalability analysis.
- Code/Algorithm Quality – Exemplary: Uses clear pseudocode and avoids duplicate pair comparisons by starting the inner loop at $i + 1$.